

Name: Mukhammadsaiid Norbaev

Student ID: NOR22597897

Module: Data Science

Coursework: Coursework 2

**Title: Student Performance Prediction
using Machine Learning**

Table of Contents

- Introduction----- 3**
- Dataset and Preprocessing Summary----- 3**
- Implementation----- 5**
 - Data Preparation----- 5
 - Train-Test Split----- 6
 - Encoding and Scaling----- 6
 - Naive Bayes Model----- 7
 - Logistic Regression Model (Extension)----- 9
 - Implementation Summary----- 12
- Reflection----- 13**
- Real-World Application----- 13**

Introduction

This report presents the implementation and evaluation of a machine learning model based on the dataset and proposal developed in Coursework 1. The main objective of this coursework is to apply a chosen technique in practice and analyse how well it solves the defined problems.

In Coursework 1, the focus was on identifying factors that influence student performance and predicting whether a student is likely to pass or fail. Naive Bayes was selected as the most appropriate method for this task due to its simplicity and effectiveness with categorical data. In this coursework, that proposed method is implemented using Python, and the results are analysed and evaluated.

In addition to implementing Naive Bayes, an additional model, Logistic Regression, is also explored in order to better understand the effect of different features on the prediction. This allows a comparison between models and provides deeper insight into the dataset.

Dataset and Preprocessing Summary

The dataset used in this coursework is the same as the one used in Coursework 1. It was obtained from Kaggle and contains over 6000 records of student-related data, including both numerical and categorical attributes such as study hours, attendance, motivation level, and exam scores.

During the preprocessing stage, several steps were applied to ensure that the dataset was clean and suitable for analysis. First, the dataset was inspected for missing values. A small number of missing values were identified in columns such as *Teacher_Quality*, *Parental_Education_Level*, and *Distance_from_Home*. These missing values were handled using **imputation**, where categorical values were replaced with the most frequent category (mode), ensuring that no data was unnecessarily removed.

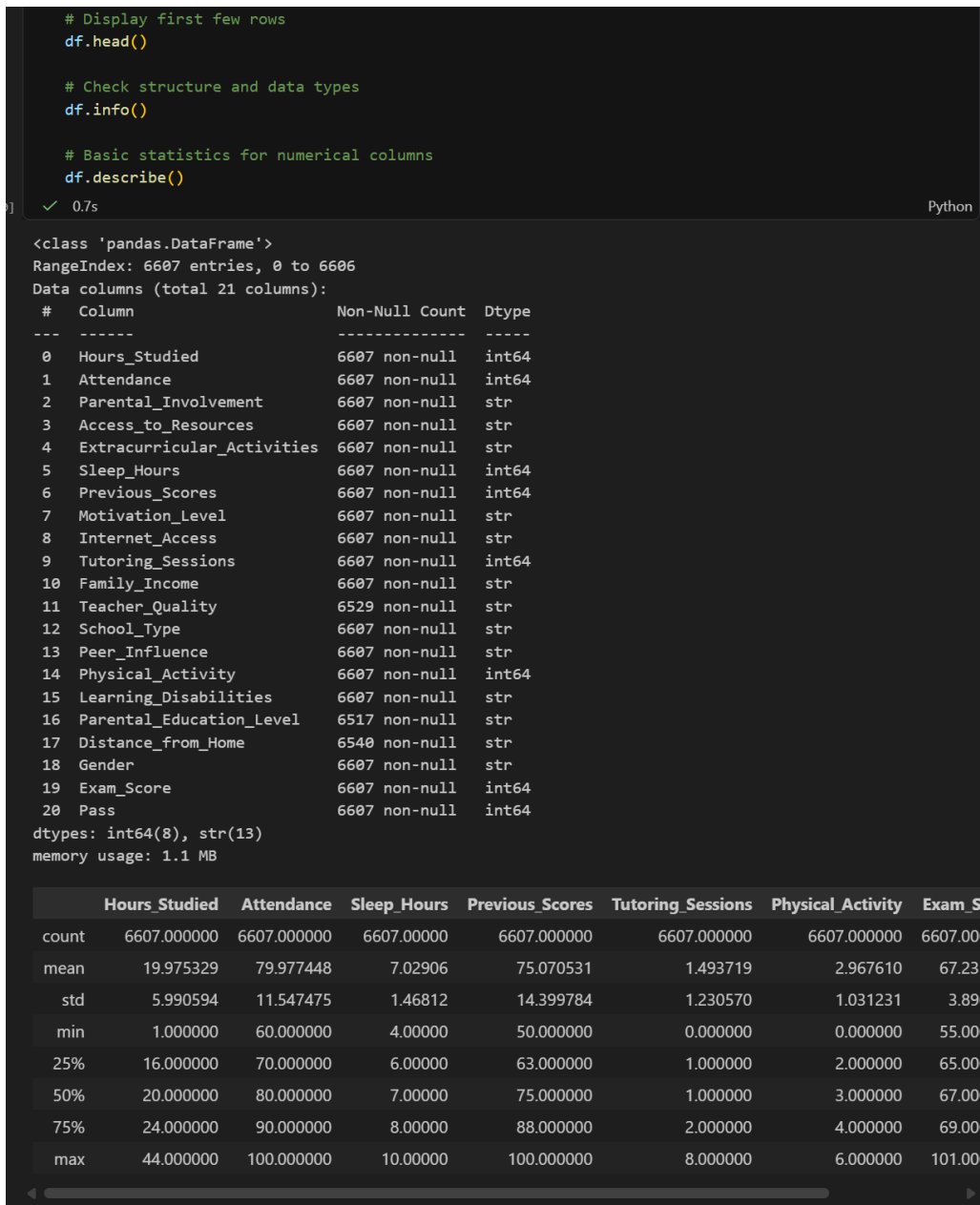
Duplicate records were also checked and removed to maintain consistency and avoid bias in the dataset. Data types were reviewed to ensure that numerical and categorical variables were correctly formatted.

One of the important features in the dataset is *Previous_Scores*, which represents the student's past academic performance. This variable is particularly useful as it provides historical context and is likely to have a strong influence on future performance.

A new variable called *Pass* was created from the original *Exam_Score* attribute. Students with a score above a certain threshold were classified as pass (1), and those below as fail (0). Initially, a threshold of 50 was used; however, during implementation, it was found that this resulted in all students being classified as pass. To address this issue and create a meaningful classification problem, the threshold was adjusted to 70, producing a more balanced dataset.

Although the original *Exam_Score* variable provides more precise numerical information, the classification approach was retained in order to remain consistent with the problem definition from Coursework 1. However, this limitation is acknowledged, as using a continuous target variable could provide more detailed insights.

Finally, feature scaling was applied using Min-Max normalisation, which transforms all numerical values into a range between 0 and 1. This ensures that features with larger magnitudes do not dominate the model and improves overall performance.



Dataset preview

```
# change threshold for passing scores
df["Pass"] = (df["Exam_Score"] >= 70).astype(int)

# update the dataset
df.to_excel("NOR22597897_CleanedDataset_v2.xlsx", index=False)

# Check distribution

df["Pass"].value_counts()
```

✓ 1.0s Python

Pass	
0	4982
1	1625

Name: count, dtype: int64

Code where *Pass* variable is redefined

Implementation

Data Preparation

The dataset was first loaded into a Python environment using the pandas library. The target variable (*Pass*) was separated from the feature variables in order to prepare the data for model training.

Before proceeding further, the *Pass* variable was redefined using a threshold of 70. This was necessary because the original threshold resulted in all students being classified as pass, which made the dataset unsuitable for classification. Adjusting the threshold created a more balanced distribution and allowed the model to learn meaningful patterns.

The original variable *Exam_Score* was removed from the feature set to avoid data leakage, as it directly determines the *Pass* variable and would otherwise lead to unrealistic model performance.

```
X = df.drop(columns=["Pass", "Exam_Score"])
y = df["Pass"]
```

✓ 0.0s Python

Pass redefinition, and X / y creation

Train-Test Split

The dataset was split into training and testing sets using an 80/20 ratio. The training set was used to train the model, while the testing set was used to evaluate its performance on unseen data.

A fixed random state was used to ensure that the results are reproducible.

```
# Split the data:
# 80% for training the model
# 20% for testing the model
# random_state ensures we get the same split every time (reproducibility)
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42
)
```

✓ 0.0s Python

Encoding and Scaling

Since the dataset contains categorical variables, these were converted into numerical format using one-hot encoding. This step is necessary because machine learning models cannot process text-based data directly.

After encoding, feature scaling was applied using Min-Max normalisation. This transforms all numerical values into a range between 0 and 1, ensuring that features with larger magnitudes do not dominate the model. Scaling was applied after encoding and before training the model, ensuring that all features contribute equally during the learning process.

This step also directly addresses feedback from Coursework 1 regarding the importance of normalising data.

```
# Convert categorical features into numerical format using one-hot encoding
# This is required because machine learning models cannot handle text directly
X_train = pd.get_dummies(X_train)
X_test = pd.get_dummies(X_test)

# Ensure both training and testing sets have the same columns
# If a category exists in one but not the other, we fill missing columns with 0
X_train, X_test = X_train.align(X_test, join='left', axis=1, fill_value=0)
```

✓ 0.0s Python

Naive Bayes Model

The Naive Bayes model was implemented as originally proposed in Coursework 1. The model was trained using the training dataset and then used to make predictions on the test dataset.

The model achieved an accuracy of approximately **88%**, indicating that it performs well in predicting whether a student will pass or fail.

However, it is important to note that Naive Bayes assumes that all features are independent. In real-world data, this assumption is often not valid, as factors such as study hours, attendance and previous scores are likely to be related. This limitation may affect the model's performance.

```
# Create scaler object
scaler = MinMaxScaler()

# Fit the scaler on training data and transform it
# (learns min/max from training data)
X_train = scaler.fit_transform(X_train)

# Apply the same transformation to test data
# (important: DO NOT fit again on test data)
X_test = scaler.transform(X_test)
```

✓ 0.0s Python

```
# Create model instance
model = GaussianNB()

# Train the model using training data
model.fit(X_train, y_train)
```

✓ 0.0s Python

▼ GaussianNB ⓘ ⓘ

▼ Parameters

priors	None
var_smoothing	1e-09

```
# Predict outcomes on test data
y_pred = model.predict(X_test)
```

✓ 0.0s Python

Naive Bayes training code

```

# Calculate accuracy (how many predictions were correct)
accuracy = accuracy_score(y_test, y_pred)
print("Accuracy:", accuracy)

# Confusion matrix shows detailed prediction results
print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred))

# report (precision, recall, F1-score)
print("Classification Report:\n", classification_report(y_test, y_pred))

```

✓ 0.0s

Python

Accuracy: 0.8812405446293494

Confusion Matrix:

```

[[890  84]
 [ 73 275]]

```

Classification Report:

	precision	recall	f1-score	support
0	0.92	0.91	0.92	974
1	0.77	0.79	0.78	348
accuracy			0.88	1322
macro avg	0.85	0.85	0.85	1322
weighted avg	0.88	0.88	0.88	1322

Accuracy output



Confusion matrix heatmap

The confusion matrix shows that the model correctly predicted a large number of failing students, but made more errors when predicting students who passed.

Logistic Regression Model (Extension)

Logistic Regression was implemented as an additional experiment, not as a replacement for the original model. The purpose of this model was to better understand the relationship between features and the target variable.

The Logistic Regression model achieved an accuracy of approximately **97.5%**, with significantly fewer misclassifications compared to Naive Bayes. This indicates that it is more effective at capturing patterns in the dataset.

One possible reason for this improved performance is that Logistic Regression does not assume feature independence, allowing it to better model relationships between variables.

In addition, Logistic Regression provides insight into the influence of individual features through its coefficients. The results show that variables such as *Hours_Studied*, *Attendance*, and *Access_to_Resources* have the strongest impact on student performance.

This directly addresses feedback from Coursework 1, where analysing the effect of variables on the target was recommended.

While Logistic Regression achieved higher accuracy, this does not necessarily mean it is universally better, as performance may vary depending on the dataset and problem complexity.

```
# Train logistic regression
# Create model instance
log_model = LogisticRegression(max_iter=1000)

# Train the model on training data
log_model.fit(X_train, y_train)
```

✓ 0.0s Python

▼ LogisticRegression ⓘ ?

► Parameters

```
# Predict outcomes on test data
y_pred_log = log_model.predict(X_test)
```

✓ 0.0s Python

```
# Evaluate Logistic Regression model
log_accuracy = accuracy_score(y_test, y_pred_log)
print("Logistic Regression Accuracy:", log_accuracy)

print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred_log))

print("Classification Report:\n", classification_report(y_test, y_pred_log))
```

✓ 0.0s Python

Logistic Regression Accuracy: 0.9750378214826021

Confusion Matrix:

```
[[969  5]
 [ 28 320]]
```

Classification Report:

	precision	recall	f1-score	support
0	0.97	0.99	0.98	974
1	0.98	0.92	0.95	348
accuracy			0.98	1322
macro avg	0.98	0.96	0.97	1322
weighted avg	0.98	0.98	0.97	1322

Logistic Regression Training and Results

```

# Get feature importance (coefficients)
feature_names = X.columns # original column names before encoding
coefficients = log_model.coef_[0]

# Create dataframe to view importance
feature_importance = pd.DataFrame({
    "Feature": feature_names,
    "Coefficient": coefficients[:len(feature_names)]
})

# Sort by importance
feature_importance = feature_importance.sort_values(by="Coefficient", ascending=False)

feature_importance.head(10)

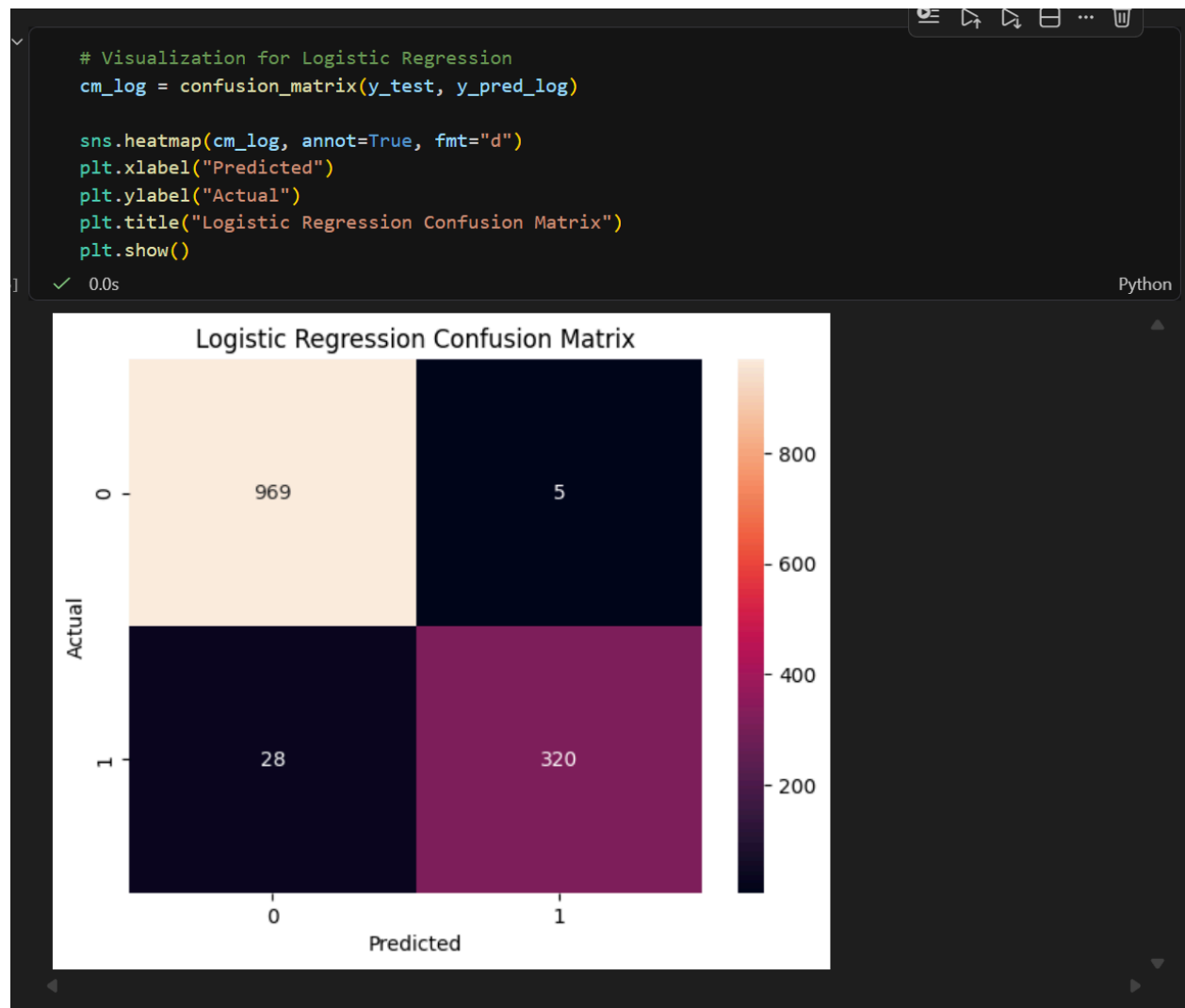
```

✓ 0.0s

Python

	Feature	Coefficient
0	Hours_Studied	14.594916
1	Attendance	10.087211
4	Extracurricular_Activities	4.624128
3	Access_to_Resources	3.129239
5	Sleep_Hours	1.579006
9	Tutoring_Sessions	1.364422
6	Previous_Scores	1.219936
14	Physical_Activity	0.729943
18	Gender	0.696960
13	Peer_Influence	0.346160

Feature importance table



Logistic Regression Confusion Matrix

The confusion matrix shows very few misclassifications, with most predictions being correct for both classes.

Implementation Summary

All models were implemented in a structured and well-documented Jupyter notebook, with clear comments and logical flow. This ensures that the implementation is reproducible and easy to understand.

Reflection

In Coursework 1, Naive Bayes was selected as the most appropriate technique for solving the problem. The results from this coursework show that this was a reasonable choice, as the model achieved good performance and was able to classify students effectively.

However, the implementation process revealed certain limitations. The assumption of feature independence in Naive Bayes does not fully reflect real-world relationships between variables, which likely reduced its overall performance.

Another issue encountered was the initial imbalance in the dataset caused by the Pass threshold. Identifying and correcting this issue was an important step, as it allowed the model to learn meaningful patterns and produce valid results.

The use of Logistic Regression provided additional insight into the dataset and helped to better understand the influence of different features. This directly addresses the feedback from Coursework 1, where it was suggested to analyse the effect of variables on the target outcome.

Although Logistic Regression achieved higher accuracy, it was used as a supporting model rather than replacing the original approach. This ensures consistency with the problem definition from Coursework 1 while still improving the overall analysis.

In terms of improvements, further work could include:

- handling class imbalance more effectively
- using regression techniques to predict *Exam_Score* directly as a continuous variable for more precise analysis
- testing additional models for comparison

Real-World Application

The results of this analysis have practical applications in the education sector. The model can be used to identify students who are at risk of failing and allow institutions to provide early intervention, such as tutoring or additional academic support.

This analysis directly addresses the problems identified in Coursework 1 by demonstrating how data-driven analysis supports better decision-making and improved student outcomes. By successfully predicting whether a student is likely to pass or fail and identifying those at risk, the model provides actionable insights into how various factors influence academic performance.